



**POLITÉCNICO
ESTELLA**

**ASIGNATURA: PROGRAMACIÓN DE SERVICIOS Y
PROCESOS**

**GRADO SUPERIOR EN DESARROLLO DE
APLICACIONES MULTIPLATAFORMA**

PSP03 Ejercicio1

Presentado por: Eugen Moga

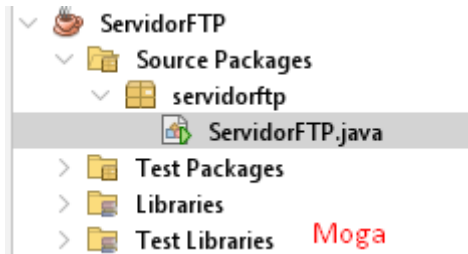
Fecha: 27/01/2026

ÍNDICE

Servidor FTP	1
Cliente FTP	4
Probando el funcionamiento.....	7

Servidor FTP

Creo el proyecto y le pongo el nombre ServidorFTP con una única clase que contiene el método main



Lo primero que hago es comprobar que se ingresa el parámetro del puerto como argumento en la ejecución y en la variable: puerto le asigno la entrada del argumento

```
9  /**
10  * @author Eugen Moga
11  * PSP Tema 3 Tarea
12  * Ejercicio 1 Lectura remota de ficheros mediante socket TCP
13  */
14  public class ServidorFTP {
15
16      /**
17       * @param args argumentos de linea de comandos
18       *      args[0] -> Puerto donde escucha */
19      public static void main(String[] args) {
20
21          if(args.length != 1){
22              System.out.println("Uso: java -jar ServidorFTP.jar <puerto>");
23              return;
24          }
25
26          int puerto = Integer.parseInt(args[0]);
27
```

Creo el ServerSocket skServidor dentro de un bloque try catch y le indico el puerto donde escucha que se introduce como argumento de ejecución. Y una vez que se ejecuta el Servidor muestro el mensaje: Escucho en el puerto + puerto.

```
28 // Creo el ServerSocket dentro de try catch para
29 // garantizar el cierre automatico del socket al finalizar
30 try (ServerSocket skServidor = new ServerSocket(puerto)){
31
32     System.out.println("Escucho en el puerto: " + puerto);
33 }
```

Creo el Socket cliente y le indico que skServidor acepte la conexión y seguido muestra el mensaje: Cliente conectado.

```
33 Socket cliente = skServidor.accept();
35 System.out.println("Cliente conectado");
36
```

Con DataInputStream creo el flujo de entrada para recibir datos del cliente. Y con DataOutputStream creo el flujo de salida para enviar datos al cliente.

```
37 // Flujo de entrada para recibir datos del cliente
38 DataInputStream entrada = new DataInputStream(cliente.getInputStream());
39
40 // Flujo de salida para enviar datos al cliente
41 DataOutputStream salida = new DataOutputStream(cliente.getOutputStream());
42
```

Espero a que el cliente indique la ruta del fichero, cuando el flujo de entrada lee la entrada del cliente muestro el mensaje: Solicitud de fichero: + rutaFichero

```
42
43 // Recibir ruta del fichero solicitado por el cliente
44 String rutaFichero = entrada.readUTF();
45 System.out.println("Solicitud de fichero: " + rutaFichero);
46
```

Con `BufferedReader` creo el buffer de lectura y le paso la ruta del fichero indicada por el cliente.

Con el bucle `while` valido que mientras el buffer de lectura siga leyendo líneas se ejecuta y con el flujo de salida envío las líneas al cliente. Cuando el buffer de lectura es nulo (ya no tiene líneas por leer) envío el mensaje "Fin".

El Buffer de lectura lo tengo puesto en un bloque `try-catch`, en caso de que no se pueda leer el fichero envía el mensaje `ERROR` y No se pudo leer el fichero.

Para finalizar le indico el cierre del `Socket` cliente

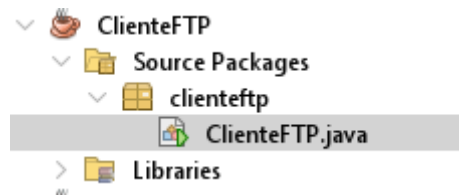
```
46
47 // Se abre y se lee el fichero
48 try {BufferedReader br = new BufferedReader(new FileReader(rutaFichero))}{
49
50     String linea;
51
52     // Cada linea se envia al cliente mediante el socket
53     while ((linea = br.readLine()) != null){
54         salida.writeUTF(linea);
55     }
56     salida.writeUTF("Fin"); // Fin de fichero
57
58 }catch (Exception e){
59     // Captura de error si no se puede leer el fichero
60     salida.writeUTF("ERROR");
61     salida.writeUTF("No se pudo leer el fichero");
62 }
63
64 // Se cierra el socket del cliente.
65 cliente.close();
66
```

Para finalizar capturo la excepción principal, por si hay algún error con el servidor.

```
66
67 }catch (Exception e){
68     // Captura de errores generales del servidor
69     System.out.println("Error servidor: " + e.getMessage());
70 }
71
```

Cliente FTP

Creo el proyecto ClienteFTP con su clase principal que contiene el método main



En el método main lo primero que hago es comprobar que se han introducido los dos parámetros de entrada al ejecutar el ClienteFTP y le asigno que el argumento 0 pertenece a la IP del servidor y el argumento 1 pertenece al puerto donde escucha el servidor.

```
11  /**
12   *
13   * @author Eugen Moga
14   *
15   * PSP Tema 3 Tarea
16   * Ejercicio 1 Cliente para la lectura remota de ficheros mediante sockets TCP
17   */
18  public class ClienteFTP {
19
20      /**
21       * @param args argumentos de línea de comandos
22       *      args[0] -> IP del servidor
23       *      args[1] -> Puerto del servidor
24       */
25      public static void main(String[] args) {
26
27          if (args.length != 2){
28              System.out.println("Uso: java -jar ClienteFTP.jar <ip_servidor> <puerto> ");
29              return;
30          }
31
32          // Se obtienen los parámetros de conexión
33          String ip = args[0];
34          int puerto = Integer.parseInt(args[1]);
35      }
```

Creo el Socket sCliente y le indico la dirección IP y el puerto del servidor todo esto dentro de un bloque try-catch para controlar las excepciones y el cierre automático del recurso.

También creo los flujos de entrada y salida con DataInputStream y DataOutputStream para poder comunicar con el servidor y poder enviar y recibir los datos.

```
35
36 // Creacion del socket del cliente y conexion con el servidor
37 // dentro de un bloque try catch para asegurar el cierre automatico
38 try(Socket sCliente = new Socket(ip, puerto)){
39
40     // Flujo de entrada y de salida
41     DataInputStream entrada = new DataInputStream(sCliente.getInputStream());
42     DataOutputStream salida = new DataOutputStream(sCliente.getOutputStream());
43
```

Creo la variable teclado de tipo Scanner para insertar datos desde el teclado y poder indicar la ruta del fichero que quiero solicitar al servidor.

Muestro el mensaje: "Introduce la ruta del fichero..." y espero a que el usuario inserte la ruta.

Seguido con el flujo de salida le envié la ruta al servidor.

```
43
44 Scanner teclado = new Scanner(System.in);
45
46 // Se indica la ruta del fichero
47 System.out.println("Introduce la ruta completa del fichero: ");
48 String ruta = teclado.nextLine();
49
50 salida.writeUTF(ruta);
51
```

Con `BufferedWriter` creo un buffer para escribir los datos que envía el servidor y con `FileWriter` creo el fichero: "fichero_recibido.txt" que es donde se va a escribir todas las líneas recibidas desde el servidor.

Utilizo el bucle `while` para indicarle que se ejecute mientras sea verdadero. Creo la variable `linea` y le asigno el flujo de entrada que llega del servidor y hago 2 comprobaciones. En la primera compruebo si la línea recibida es igual a "Fin", si se cumple se sale del bucle. En la segunda comprobación compruebo si la línea recibida es igual a "ERROR" y muestro el mensaje de error y salgo del bucle.

Si ninguna de estas comprobaciones se cumple. Con el `BufferedWriter` escribo la línea en el fichero creado anteriormente.

```
51 // Se crea el fichero local para guardar el contenido recibido
52
53 try {BufferedWriter bw = new BufferedWriter(new FileWriter("fichero_recibido.txt")){
54     while (true){
55         String linea = entrada.readUTF();
56
57         if (linea.equals("Fin")){
58             break;
59         }
60
61         if (linea.equals("ERROR")){
62             System.out.println("Error al leer el fichero en el servidor ");
63             System.out.println(entrada.readUTF());
64             return;
65         }
66
67         // Se escribe la linea recibida en el fichero local
68         bw.write(linea);
69         bw.newLine();
70     }
71 }
```

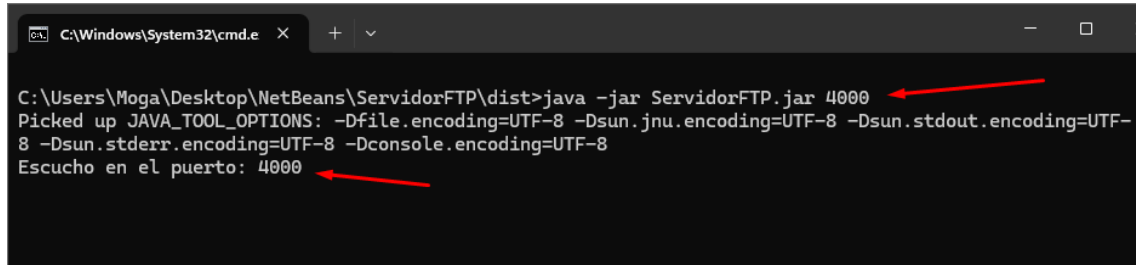
Para finalizar cierro el bloque `try` abierto para controlar el `Socket`

```
74
75 }catch(Exception e){
76     // Se captura errores de conexion.
77     System.err.println("Error cliente: " + e.getMessage());
78 }
```


Probando el funcionamiento

Para probar que funciona primero lanzo en servidor desde la terminal con el comando: `java -jar ServidorFTP.jar 4000`

Y se puede ver en la captura como el servidor muestra el mensaje que esta escuchando en el puerto 4000

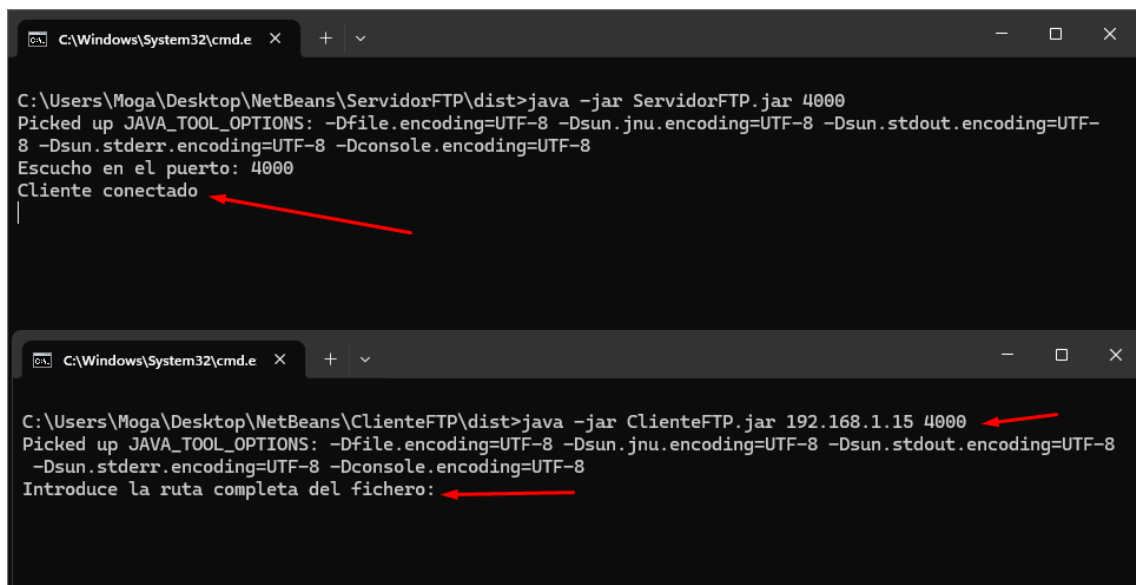


```
C:\Windows\System32\cmd.e  X  +  v  -  □  X
C:\Users\Moga\Desktop\NetBeans\ServidorFTP\dist>java -jar ServidorFTP.jar 4000
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
-Dsun.stderr.encoding=UTF-8 -Dconsole.encoding=UTF-8
Escucho en el puerto: 4000
```

Paso a ejecutar el cliente FTP con el comando:

`java -jar ClienteFTP.jar 192.168.1.15 4000`

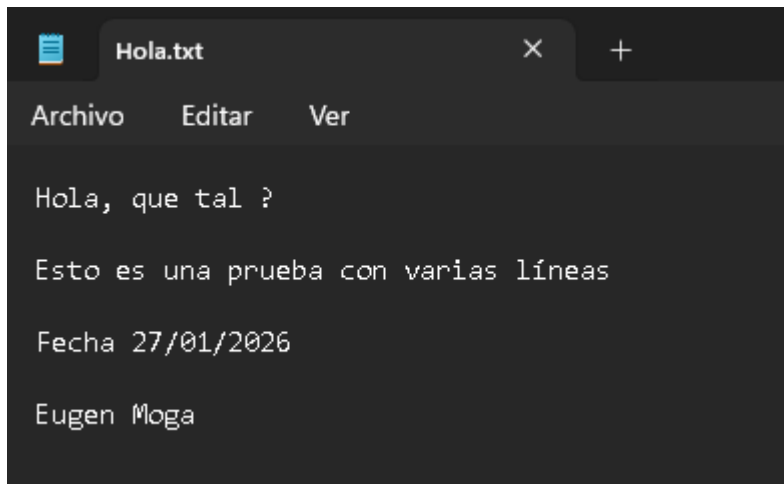
En la captura se puede ver que una vez ejecutado el servidor muestra el mensaje: Cliente conectado y en el cliente solicita el que Introduzca la ruta del fichero:



```
C:\Windows\System32\cmd.e  X  +  v  -  □  X
C:\Users\Moga\Desktop\NetBeans\ServidorFTP\dist>java -jar ServidorFTP.jar 4000
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
-Dsun.stderr.encoding=UTF-8 -Dconsole.encoding=UTF-8
Escucho en el puerto: 4000
Cliente conectado

C:\Windows\System32\cmd.e  X  +  v  -  □  X
C:\Users\Moga\Desktop\NetBeans\ClienteFTP\dist>java -jar ClienteFTP.jar 192.168.1.15 4000
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
-Dsun.stderr.encoding=UTF-8 -Dconsole.encoding=UTF-8
Introduce la ruta completa del fichero:
```

Tengo un fichero de prueba en el disco E:\ que tiene el nombre Hola.txt que tiene el siguiente contenido.

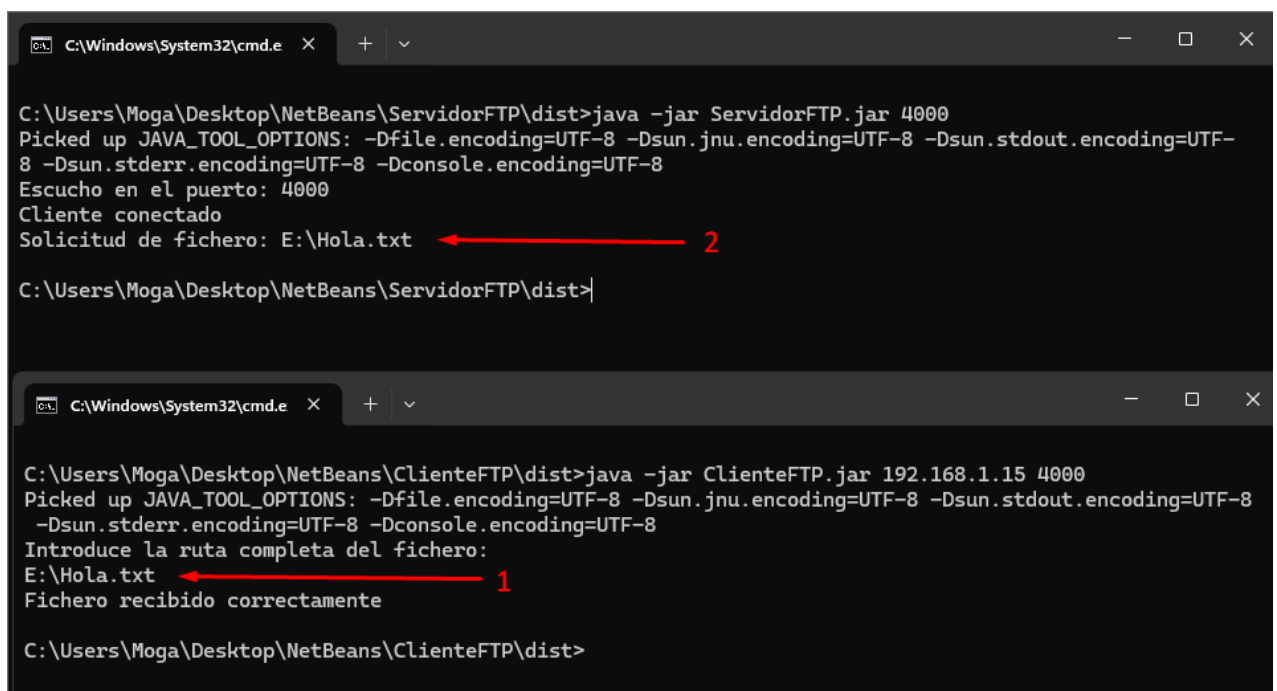


A screenshot of a text editor window titled 'Hola.txt'. The window has a menu bar with 'Archivo', 'Editar', and 'Ver'. The text content is as follows:

```
Hola, que tal ?  
  
Esto es una prueba con varias líneas  
  
Fecha 27/01/2026  
  
Eugen Moga
```

En el ClienteFTP le indico la ruta del fichero que tiene que leer el servidor:

1. Indico la ruta E:\Hola.txt
2. El servidor muestra el mensaje con la solicitud del fichero y finaliza la ejecución del servidor.



Two screenshots of Windows command prompts. The top screenshot shows the execution of the 'ServidorFTP.jar' program. The bottom screenshot shows the execution of the 'ClienteFTP.jar' program.

Top Screenshot (ServidorFTP.jar):

```
C:\Windows\System32\cmd.e X + v  
C:\Users\Moga\Desktop\NetBeans\ServidorFTP\dist>java -jar ServidorFTP.jar 4000  
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -Dconsole.encoding=UTF-8  
Escucho en el puerto: 4000  
Cliente conectado  
Solicitud de fichero: E:\Hola.txt ← 2  
C:\Users\Moga\Desktop\NetBeans\ServidorFTP\dist>
```

Bottom Screenshot (ClienteFTP.jar):

```
C:\Windows\System32\cmd.e X + v  
C:\Users\Moga\Desktop\NetBeans\ClienteFTP\dist>java -jar ClienteFTP.jar 192.168.1.15 4000  
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8 -Dsun.jnu.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -Dconsole.encoding=UTF-8  
Introduce la ruta completa del fichero:  
E:\Hola.txt ← 1  
Fichero recibido correctamente  
C:\Users\Moga\Desktop\NetBeans\ClienteFTP\dist>
```

En la carpeta donde esta el archivo ClienteFTP.jar se ha creado el fichero fichero_recibido.txt donde se puede ver que se ha copiado correctamente el fichero desde el servidor al cliente

